CHAPTER 3

Unknown Attribute Values

The basic algorithm for constructing decision trees, described in the previous chapter, contains a hidden assumption: that the outcome of a test for any case can be determined. This assumption comes into play when partitioning a set T of training cases into subsets $\{T_i\}$ associated with the outcomes of the selected test, and in a similar implicit partitioning during the evaluation of possible tests. Further, classifying a case using a decision tree, as outlined in Section 1.1, requires taking the branch appropriate to the case at each decision node. Since every test is based on a single attribute, the outcome of a test cannot be determined unless the value of that attribute is known.

It is an unfortunate fact of life that data often has missing attribute values. This might occur because the value is not relevant to a particular case, was not recorded when the data was collected, or could not be deciphered by the person responsible for putting the data into machine-readable form. For example, I have carried out numerous experiments with thyroid assay data from the Garvan Institute of Medical Research. Each case concerns one patient referral and includes information provided by the referring physician as well as the results of requested laboratory measurements. In one batch of three thousand cases, many cases failed to specify the age and/or sex of the patient, even though this information is usually relevant to interpretation; this is presumably an example of incomplete data collection. More than 30% of the cases were missing some information, usually assays that were not requested and so were probably irrelevant to interpretation. Such incompleteness is typical of real-world data.

We are then left with a choice: Either a significant proportion of available data must be discarded and some test cases pronounced unclassifiable, or the algorithms must be amended to cope with missing attribute values. In most situations the former course is unacceptable as it weakens the ability to find patterns. To address the second alternative, three questions must be answered:

- Selection of a test on which to partition the training set is made on the basis of heuristic criteria such as *gain* or *gain ratio*. If two tests

use attributes with different numbers of unknown values, how should this be taken into account when weighing their relative desirability?

- Once a test has been selected, training cases with unknown values of the relevant attribute cannot be associated with a particular outcome of the test, and so cannot be assigned to a particular subset T_i. How should these cases be treated in the partitioning?

- When the decision tree is used to classify an unseen case, how should the system proceed if the case has an unknown value for the attribute tested in the current decision node?

Several authors have developed different answers to these questions, usually based either on filling in missing attribute values (with the most probable value or a value determined by exploiting interrelationships among the values of different attributes), or on looking at the probability distribution of values of attributes. More common approaches are compared empirically in Quinlan [1989], leading to the general conclusion that some approaches are clearly inferior, but no one approach is uniformly superior. The treatment of missing attribute values in C4.5 that is described below has proved satisfactory for the sort of data that I use, but several alternative schemes employed by other systems are probably just as good.

3.1 Adapting the previous algorithms

3.1.1 Evaluating tests

Modification of the criteria for choosing tests is relatively straightforward because they are derived from the notion of information. Recall that the *gain* of a test measures the information about class membership that can be expected from partitioning the training cases, calculated by subtracting the expected information required to ascertain the class of a case after the partition from the same quantity before partition. It is evident that a test can provide no information about the class membership of cases whose value of the test attribute is unknown.

As in Section 2.2, let T be a training set and X a test based on some attribute A, and suppose that the value of A is known in fraction F of the cases in T. Let $info(T)$ and $info_X(T)$ be calculated as before, except that only cases with known values of A are taken into account. The

definition of *gain* can reasonably be amended to

$$\begin{aligned} gain(X) &= \text{probability } A \text{ is known} \times (info(T) - info_X(T)) \\ &\quad + \text{probability } A \text{ is not known} \times 0 \\ &= F \times (info(T) - info_X(T)) \end{aligned}$$

or, in other words, the apparent gain from looking at cases with known values of the relevant attribute, multiplied by the fraction of such cases in the training set.

Similarly, the definition of *split info*(X) can be altered by regarding the cases with unknown values as an additional group. If a test has n outcomes, its split information is computed as if the test divided the cases into $n + 1$ subsets.

3.1.2 Partitioning the training set

A test can be selected from the possible tests as before, using the modified definitions of *gain* and *split info*. If the test X with outcomes $O_1, O_2, \ldots, O_n$ that is finally chosen has unknown outcomes on some of the training cases, the concept of partitioning must be generalized.

In this situation, C4.5 follows ASSISTANT [Cestnik *et al.*, 1987] in adopting a probabilistic approach. When a case from T with known outcome O_i is assigned to subset T_i, this indicates that the probability of that case belonging in subset T_i is 1 and in all other subsets is 0. When the outcome is not known, however, only a weaker probabilistic statement can be made. We therefore associate with each case in each subset T_i a *weight* representing the probability that the case belongs to each subset. If the case has a known outcome, this weight is 1; if the case has an unknown outcome, the weight is just the probability of outcome O_i at this point. Each subset T_i is now a collection of possibly fractional cases so that $|T_i|$ should be reinterpreted as the sum of the fractional weights of the cases in the set.

Of course, the training cases in T might have had nonunit weights to start with, since T might be one subset of an earlier partition. So, in general, a case from T with weight w whose outcome is not known is assigned to each subset T_i with weight

$$w \times \text{probability of outcome } O_i.$$

The latter probability is estimated as the sum of the weights of cases in T known to have outcome O_i, divided by the sum of the weights of the cases in T with known outcomes on this test.

3.1.3 Classifying an unseen case

A similar approach is taken when the decision tree is used to classify a case. If a decision node is encountered at which the relevant attribute value is unknown, so that the outcome of the test cannot be determined, the system explores all possible outcomes and combines the resulting classifications arithmetically. Since there can now be multiple paths from the root of a tree or subtree to the leaves, a "classification" is a class distribution rather than a single class. When the total class distribution for the unseen case has been established in this way, the class with the highest probability is assigned as "the" predicted class.

3.2 Play/Don't Play example again

To clarify the effect of these modifications, we return to the training set of Figure 2–1 and suppose that the value of the attribute outlook for the case

outlook = overcast, temperature = 72°, humidity = 90%, windy = true

were unknown (denoted by "?") rather than overcast. When we restrict our attention to the 13 remaining cases whose value for outlook is known, we get the following frequencies:

	Play	Don't Play	Total
outlook = sunny	2	3	5
overcast	3	0	3
rain	3	2	5
Total	8	5	13

Using the corresponding relative frequencies to compute the quantities relevant to the assessment of a test on outlook, we have

$$
\begin{aligned}
\mathit{info}(T) &= -8/13 \times \log_2(8/13) - 5/13 \times \log_2(5/13) \\
&= 0.961 \text{ bits} \\
\mathit{info}_X(T) &= 5/13 \times (-2/5 \times \log_2(2/5) - 3/5 \times \log_2(3/5)) \\
&\quad + 3/13 \times (-3/3 \times \log_2(3/3) - 0/3 \times \log_2(0/3)) \\
&\quad + 5/13 \times (-3/5 \times \log_2(3/5) - 2/5 \times \log_2(2/5)) \\
&= 0.747 \text{ bits} \\
\mathit{gain}(X) &= 13/14 \times (0.961 - 0.747) \\
&= 0.199 \text{ bits.}
\end{aligned}
$$

The gain for this test is slightly lower than the previous value of 0.246 bits. The split information, however, is still determined from the entire training set and is larger, since there is an extra category for unknown outcome that has to be taken into account:

$$\begin{aligned} &-5/14 \times \log_2(5/14) \quad \text{(for sunny)} \\ &-3/14 \times \log_2(3/14) \quad \text{(for overcast)} \\ &-5/14 \times \log_2(5/14) \quad \text{(for rain)} \\ &-1/14 \times \log_2(1/14) \quad \text{(for "?")} \end{aligned}$$

which comes to 1.809 bits instead of the previous 1.577 bits. The gain ratio value thus drops from 0.156 to 0.110.

When the 14 training cases are partitioned by this test, the 13 cases for which the value of outlook are known present no problem. The remaining case is assigned to *all* blocks of the partition, corresponding to outlooks sunny, overcast and rain, with weights 5/13, 3/13, and 5/13 respectively.

We now focus on the first such subset of cases corresponding to outlook = sunny, namely

Outlook	Temp (°F)	Humidity (%)	Windy?	Decision	Weight
sunny	75	70	true	Play	1
sunny	80	90	true	Don't Play	1
sunny	85	85	false	Don't Play	1
sunny	72	95	false	Don't Play	1
sunny	69	70	false	Play	1
?	72	90	true	Play	5/13

If this subset is partitioned further by the same test on humidity as in Figure 2–2, the class distributions in the subsets are

humidity ≤ 75	2 class Play,	0 class Don't Play
humidity > 75	5/13 class Play,	3 class Don't Play

The first consists of cases of the single class Play. The second still contains cases from both classes but the program can find no test that sensibly improves on the situation. Similarly, the subset corresponding to outlook = rain cannot be partitioned into single-class subsets. The decision tree has much the same structure as before,

```
outlook = sunny:
|   humidity ≤ 75: Play (2.0)
|   humidity > 75: Don't Play (3.4/0.4)
outlook = overcast: Play (3.2)
outlook = rain:
|   windy = true: Don't Play (2.4/0.4)
|   windy = false: Play (3.0)
```

but the numbers at the leaves, of the form (N) or (N/E), now become important. N is the sum of the fractional cases that reach the leaf; in the second form, E is the number of cases that belong to classes other than the nominated class.[4] So

Don't Play (3.4/0.4)

means that 3.4 (fractional) training cases reached this leaf, of which 0.4 did not belong to class Don't Play.

We now investigate what happens when this tree is used to classify another case with sunny outlook, temperature 70°, unknown humidity, and windy false. The outlook value ensures this case moves to the first subtree, but it is not possible now to determine whether humidity ≤ 75. However, we can see that

- if the humidity were less than or equal to 75%, the case would be classified as Play, and
- if the humidity were greater than 75%, the case would be classified as Don't Play with probability 3/3.4 (88%) and Play with probability 0.4/3.4 (12%).

At the time the tree was constructed, the partitions for these outcomes contained 2.0 and 3.4 cases respectively. The conditional conclusions are combined with the same relative weights, 2.0/5.4 and 3.4/5.4, so that the final class distribution for this case is

Play:	$2.0/5.4 \times 100\% + 3.4/5.4 \times 12\% = 44\%$
Don't Play:	$3.4/5.4 \times 88\% = 56\%$

3.3 Recapitulation

It is possible to get enmeshed in details of calculations like these, thereby losing sight of the fundamental simplicity of the approach. C4.5 assumes

4. E has a slightly different meaning for a pruned tree, as explained in the next chapter.

that unknown test outcomes are distributed probabilistically according to the relative frequency of known outcomes. A (possibly fractional) case with an unknown test outcome is divided into fragments whose weights are proportional to these relative frequencies, with the result that a single case can follow multiple paths in the tree. This applies equally when the training cases are divided during the construction of a tree and when the tree is used to classify cases.

It is interesting to compare this approach with that taken in CART. Whenever a test is chosen, CART finds another test called a *surrogate split* whose partition of the training cases is most similar to that of the original test. If a case's outcome of the original test is not known, the outcome of the surrogate is used instead; if this is unknown too, the next best surrogate is invoked, and so on. This method does not call for individual cases to be fragmented and so is more efficient. It depends, however, on being able to find suitably accurate surrogate splits, which seems unlikely in domains where tests have different numbers of outcomes and where there is little correlation between attributes.